

Ultra Fast Packet Crafter / Ping Utility: Blitzping (DPDK Port and Traceroute)

Anurag Singh Arjun B Dikshit Srivaths P Tanish Yelgoe
Siddharth Rajandekar

Indian Institute of Technology Gandhinagar

November 14, 2025

Outline

- 1 Introduction
- 2 Methodology
- 3 Results
- 4 Conclusion

Introduction

- Modern high-speed networks demand tools that can operate at line rate with minimal system overhead.
- Traditional utilities suffer from kernel bottlenecks like context switches and buffer copies.
- **Blitzping** is a high-performance packet crafter and probing tool.
- Our project extends Blitzping with:
 - ① **DPDK-based kernel-bypass transmission.**
 - ② **TTL-based traceroute** with real-time RTT measurement.

Porting Blitzping to DPDK

- Replaced POSIX sockets with **DPDK** for zero-copy packet I/O.
- Initialization:
 - Check if `--use-dpdk` flag exists.
 - Initialize Environment Abstraction Layer (EAL).
 - Configure NIC with TX/RX queues using `rte_eth_dev_configure`.
- Transmission:
 - Allocated packet buffers from `rte_mbuf` pools.
 - Used `rte_eth_tx_burst()` for high-speed send loop.
 - Offloaded checksum computation to NIC hardware.

DPDK Send Loop and Optimization

- Randomized source IP and port for each packet.
- Used high-resolution `rte_get_tsc_cycles()` for timing.
- Minimal per-packet CPU work.
- Zero system calls and kernel context switching.

Key Benefit

Achieved **4.5× higher throughput** compared to POSIX sockets.

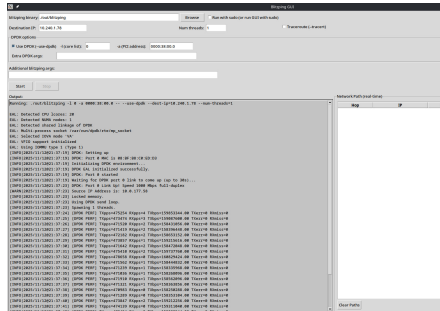
Key DPDK Commands Used

- `rte_eal_init(argc, argv)`
 - Initializes the DPDK Environment Abstraction Layer.
 - Sets up hugepages, memory pools, and CPU cores.
- `rte_eth_dev_configure(port_id, rx_queues, tx_queues, &conf)`
 - Configures the Ethernet device with RX/TX queues and offload features.
- `rte_eth_rx_queue_setup()` / `rte_eth_tx_queue_setup()`
 - Allocate and initialize RX/TX queues for each core or thread.
- `rte_eth_dev_start(port_id)`
 - Starts the Ethernet port for packet transmission and reception.
- `rte_pktmbuf_alloc(mempool)`
 - Allocates an mbuf (packet buffer) from a pre-created memory pool.
- `rte_eth_tx_burst(port_id, queue_id, &mbuf, 1)`
 - Sends one or more packets directly to the NIC for transmission.
- `rte_eth_stats_get(port_id, &stats)`
 - Retrieves NIC statistics like TX/RX packets per second.

Performance Comparison

Configuration	TXpps (avg)	TXbps (avg)	Observation
Normal (1 thread)	105k	23.7 Mbps	Kernel-limited
Normal (20 threads)	5.8k	1.2 Mbps	Contention overhead
DPDK (1 thread)	472k	159 Mbps	4.5 $\times$ gain
DPDK (20 threads)	14k	4.8 Mbps	Queue conflicts

Original Blitzping Output



DPDK Enhanced Output

Traceroute Implementation

- Sends UDP probes with incrementing TTL values.
- Receives ICMP replies (“Time Exceeded” or “Destination Unreachable”).
- Matches ICMP replies to probes via encoded UDP source port tokens.
- Computes RTT using `CLOCK_MONOTONIC`.

Result: Accurate per-hop latency and IP discovery of routers in real time.

Traceroute Results

- Destination: Google DNS (8.8.8.8)
- 11 hops discovered, RTT 1.2 ms to 15.3 ms.

The screenshot shows the Blitzping GUI with the following configuration:

- Blitzping binary: /usr/blitzping
- Destination IP: 8.8.8.8
- Num threads: 1
- Traceroute (-tracert): checked
- DPMK options: Use DPMK (-use-dpmk): unchecked, -i (core list): 0, -a (PCI address): 0000:28:00:0
- Extra DPMK args: (empty)
- Additional Blitzping args: (empty)

The output shows the following command and results:

```
Running: /usr/blitzping --dest-ip=8.8.8.8 --num-threads=1 --tracert
```

Network Path (real-time)

Hop	IP	Last (ms)	Avg (ms)	Count
1	10.240.0.2	4.000	4.000	1
2	10.3.0.29	2.000	2.000	1
3	10.3.0.5	4.000	4.000	1
4	172.16.4.7	2.000	2.000	1
5	14.139.90.1	3.000	3.000	1
6	10.117.81.253	2.000	2.000	1
7	10.154.8.127	11.000	11.000	1
8	10.255.239.170	11.000	11.000	1
9	10.152.7.214	14.000	14.000	1
12	8.8.8.8	17.000	17.000	1

Process exited with return code 0

Conclusion

- DPDK integration enabled **4.5× throughput gain**.
- Traceroute feature added hop-by-hop network visibility.
- Demonstrated trade-offs:
 - Kernel sockets: simpler but slower.
 - DPDK: faster but more complex.

Thank You!

Questions?